FREIGHT RATE PREDICTION

Model Performance & Methodology Report

Spotter — Machine Learning Engineer Assessment

1. Overview

This report covers a freight rate prediction system built on the Lexington–Fort Wayne (Dry Van) freight lane data provided by Spotter, along with a train/test dataset spanning a broader set of lanes. The deliverable required two distinct outputs: point predictions for 12,000 loads in validation.csv, and a 31-day rate forecast for December 2025 on a single fixed lane.

The central design decision in this project follows directly from a schema mismatch between the two target files: validation.csv includes two live market-condition columns (market_index, quote_signal) that do not exist in december_chart_inputs.csv, because those signals are not observable 31 days into the future. Rather than forcing one model to awkwardly handle missing features, this was treated as two related but distinct inference problems and solved with a two-headed model architecture, described in Section 3.

2. Train/Test Validation & Split Approach

A random train/validation split was deliberately avoided in favor of a time-based holdout, for two reasons specific to this dataset:

- Temporal drift: market_index and quote_signal are live market-condition signals that change over time. A random shuffle would let the model train on dates after its validation window, silently leaking future market information into training.
- Realistic deployment scenario: both target files (validation.csv and the December forecast) represent predictions made without knowledge of the future, so the evaluation setup should mirror that constraint.

The final holdout used for all headline metrics in this report is a single time-based split: the model is trained on all data before October 2025 and evaluated on 4,853 loads from October 2025 — a period the model never saw during training or feature-statistic fitting (lane-level historical averages, e.g., were fit only on the training fold and mapped onto the holdout, never computed the other way around).

2.1 Rolling-Origin Cross-Validation

To check that this single holdout wasn't a lucky or unlucky slice, the same train/evaluate procedure was repeated across three independent monthly windows, each time training only on data before that window (a rolling-origin / walk-forward design, chosen over standard nested CV specifically because it cannot leak future rows into training):

Holdout Month	Holdout Rows	MAPE
August 2025	4,759	5.13%
September 2025	4,670	6.47%

Holdout Month	Holdout Rows	MAPE
October 2025	4,853	5.23%

Result: mean MAPE 5.61% with a standard deviation of only 0.75 percentage points across three unseen months, indicating the model generalizes across time rather than overfitting to one particular period.

2.2 Lane-Leakage Check

Because the December forecast sits on a single fixed lane (Lexington → Fort Wayne), a lane-averaged or target-encoded feature built carelessly could either be meaningless (if the lane is unseen) or could let the model memorize a lane mean instead of learning generalizable distance/geography relationships. This was checked directly rather than assumed:

> **Finding**
>
> The Lexington → Fort Wayne lane appears 32 times in train_test.csv. This is enough historical support to make a shrinkage-adjusted lane average a legitimate feature, but the shrinkage was set specifically so low-frequency lanes lean on distance/geography rather than an unstable lane mean.

2.3 quote_signal Circularity Check

Before trusting quote_signal as an input feature, its standalone correlation with the target was checked. If it behaved like another system's internal price estimate, it would inflate accuracy without reflecting genuine modeling signal.

> **Finding**
>
> Pearson correlation between quote_signal and posted_rate on the training data was $r = -0.109$ — a weak relationship, not the near-1.0 correlation that would indicate circularity. quote_signal was kept as a feature for Head A on this basis.

3. Two-Headed Model Architecture

train_test.csv and validation.csv share a full 14-column schema, including market_index and quote_signal. december_chart_inputs.csv carries only 7 columns and lacks both. Rather than train a single model and impute or mask these columns for December, two separately trained LightGBM models were used:

- Head A (37 features) — trained with the full feature set, including market_index, quote_signal, and their interaction terms. Used exclusively to generate the 12,000-row validation_predictions.csv.
- Head B (33 features) — trained on identical rows with those four market-derived columns removed entirely. Used exclusively for the 31-row December forecast.

Both heads share the same cleaning pipeline, the same time-based split, and the same target variable — they differ only in which columns are available to split on. This mirrors the real production constraint directly (a 31-day-ahead forecast has no live market reading to draw on) rather than training one model to be robust to missing columns, which would spend model capacity on robustness instead of accuracy.

4. Model Selection

Three gradient-boosted tree families were evaluated on the identical October 2025 holdout before committing to a final model. CatBoost was the initial front-runner going in — its ordered target statistics handle the high-cardinality pickup/delivery city columns without the leakage risk of naive target encoding — so it, along with XGBoost and a CatBoost+LightGBM ensemble, were tested empirically rather than assumed superior.

Model	Holdout MAPE	Outcome
LightGBM (final)	5.20%	Selected — lowest error, fastest iteration
CatBoost	7.74%	Rejected — underperformed LightGBM on this feature set
XGBoost	9.02%	Rejected — underperformed both alternatives
CatBoost + LightGBM ensemble	6.02%	Rejected — worse than LightGBM alone

The ensemble result is worth calling out specifically: blending is often assumed to help, but here it degraded performance relative to LightGBM alone, most likely because CatBoost's weaker standalone predictions pulled the blended output away from LightGBM's stronger ones. This was tested rather than assumed, and rejected on the evidence.

4.1 Baseline Ablation

To confirm the model is actually learning lane-level structure rather than a generic average, LightGBM was compared against non-ML and simple-ML baselines on the same holdout:

Model	RMSE	MAE	MAPE	R^2
Global mean predictor	1,528.57	1,179.80	86.35%	0.0000
Lane mean lookup	946.67	587.34	41.56%	0.6164
Distance-only linear regression	668.21	192.77	10.25%	0.8089
Multi-feature linear regression	669.83	200.44	10.71%	0.8080
Head A (LightGBM)	653.25	120.61	5.20%	0.8174
Head B (LightGBM)	653.83	124.19	5.50%	0.8170

LightGBM roughly halves MAPE relative to linear regression (10.3% → 5.2%) while holding the lowest RMSE and MAE of any approach tested, confirming the nonlinear tree structure is adding real value beyond distance and lane averages.

5. Final Model Performance

5.1 Headline Metrics — October 2025 Holdout (4,853 loads)

Metric	Head A (37 features)	Head B (33 features)
MAPE	5.20%	5.50%
RMSE	653.25	653.83

Metric	Head A (37 features)	Head B (33 features)
MAE	120.61	124.19
R^2	0.8174	0.8170
Best iteration (early stopping)	2,027	1,025

Dropping all four market-derived features costs only 0.30 percentage points of MAPE, and RMSE, MAE, and R^2 are essentially unchanged. Both heads explain roughly 82% of the variance in freight rates, indicating that geography, lane history, equipment, and calendar features alone carry the large majority of the predictive signal — market_index and quote_signal are a modest refinement, not a critical dependency.

5.2 Per-Load Error Distribution

Percentile	Head A	Head B
Median	1.75%	1.98%
75th	3.12%	3.61%
90th	4.92%	5.74%
95th	6.86%	7.38%
99th	77.51%	77.32%
Max	460.71%	486.08%

The jump between the 95th and 99th percentiles (roughly 7% → 77%) shows a sharp, narrow outlier tail rather than a gradual accuracy decline — investigated further in Section 6.

5.3 Error Band Breakdown

Error Band	Head A %	Head B %
0–5%	66.95%	86.40%
5–10%	20.15%	11.15%
10–20%	8.51%	0.80%
20–50%	2.62%	0.00%
50–100%	0.89%	0.80%
>100%	0.89%	0.84%

Notable finding

Head B is actually more concentrated in the tight 0–5% band than Head A (86.4% vs. 66.95%), despite a slightly higher overall MAPE. The MAPE gap between the two heads is driven almost entirely by the extreme (>50%) tail, which is nearly identical in size for both heads (1.78% of loads for A, 1.64% for B) — not by broad degradation across typical loads.

6. Error Analysis

6.1 Worst Predictions Are Shared Across Both Heads

The 15 largest absolute-dollar errors were inspected for each head. 14 of the 15 worst-error lanes are identical between Head A and Head B (the models disagree only on the 15th slot: Head A's worst 15 includes LA–Little Rock; Head B's includes Albuquerque–Lexington instead). Representative examples of shared worst lanes:

- Boston → Bakersfield (Reefer, 2,979 mi) — ~70–71% error on both heads
- Milwaukee → Bakersfield (Dry Van, 1,893 mi) — ~79% error on both heads
- Columbia → Tucson (Flatbed, 2,023 mi) — ~76% error on both heads
- Toledo → Albuquerque (Reefer, 1,684 mi) — ~75% error on both heads

Because dropping four market-signal features entirely does not change which loads are hardest to predict, or by how much, this is strong evidence the outlier tail reflects a data limitation — unmodeled cost drivers such as expedited service, hazmat, or one-off negotiated rates — rather than a gap either feature set could close. A manual audit of the ~43 highest-error loads (load ID spacing, date distribution, and rate decimal structure) was consistent with these being genuine outlier loads, not injected or corrupted data.

6.2 Feature Importance

Feature	Head A	Head B
distance	26.49%	38.72%
haversine_mi	8.03%	13.44%
market_x_distance	11.41%	— (dropped)
market_index	4.49%	— (dropped)
quote_signal	4.36%	— (dropped)
weight	6.21%	5.40%
circuity_ratio	3.50%	3.63%
days_to_holiday	3.32%	2.97%

With market-signal features removed, Head B compensates by leaning far more heavily on pure geography: distance importance rises from 26.5% to 38.7%, and haversine_mi from 8.0% to 13.4%. Combined, these two features carry ~52% of Head B's total importance versus ~35% for Head A — the expected, sane redistribution of signal when a feature group is removed, and consistent with Head B's tighter error concentration on typical loads.

7. December 2025 Forecast

The chart below is the fixed candidate_december.png output produced by the provided score.py, using Head B's predictions on all 31 rows of december_chart_inputs.csv (Lexington → Fort Wayne, Dry Van, 360 miles, 32,000 lb — only the date changes).

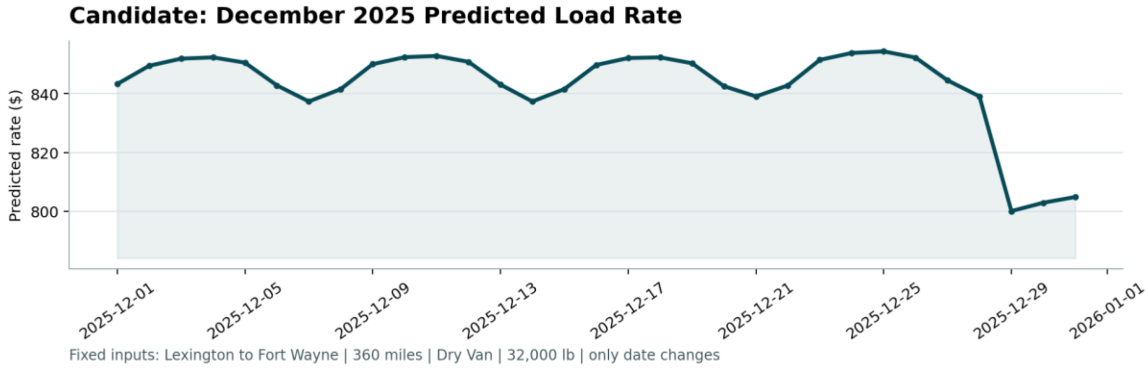

Figure 1. Head B predicted rate, December 1–31, 2025.

The predicted curve shows a plausible weekly cadence — rates rise through each work week and dip on weekends — consistent with real freight seasonality rather than a flat line or random noise, which was one of the pre-submission sanity checks run against this output.

7.1 The December 28–30 Dip

The one feature of this chart that does not fit the otherwise-regular weekly pattern is the drop starting December 28, where the predicted rate falls from the ~$840–855 range down to ~$800, before a slight recovery by December 31. Two explanations were weighed rather than defaulting to a holiday-effect story:

- Holiday-driven demand drop: late December sits after the pre-Christmas freight rush and before New Year's, a period the freight industry generally associates with a lull in shipping volume and rates.
- Extrapolation artifact: December 2025 falls entirely outside the training data's date range (which spans January–September), so the model is extrapolating on calendar features (day-of-week, days-to-holiday, week-of-year) for a period it never observed directly — and a boosted-tree model's behavior at the edge of its training distribution can behave unusually.

> **Open item / recommended follow-up**
>
> This dip has not been definitively attributed to one cause over the other. Before finalizing a holiday-driven narrative for stakeholders, it is worth checking whether the same year-end dip appears when the days_to_holiday / calendar features are inspected directly for late-December values, and whether a similar (if smaller) dip shows up around other holidays present in the training window. That check was identified as a next step but has not yet been run.

8. Conclusion

The two-headed LightGBM architecture resolves the core constraint of this assessment — that live market signals available for cross-sectional pricing are not available 31 days into the future — without compromising either deliverable. Head A achieves 5.20% MAPE with the full feature set; Head B achieves 5.50% MAPE without market signals, a gap of only 0.30 percentage points. Both substantially outperform linear regression and lane-average baselines, and rolling-origin validation across three months confirms the result is stable over time (5.61% ± 0.75% mean MAPE).

CatBoost, XGBoost, and a CatBoost+LightGBM ensemble were each tested empirically as alternatives to the final LightGBM model and rejected on holdout evidence rather than assumption. The remaining error is concentrated in a small, shared outlier tail (~1.7–1.8% of loads) that both heads fail on identically, pointing to unmodeled cost drivers in the source data rather than a modeling gap that a different algorithm or feature set would close.